

Testing Theory and Practice 101

Dr Chris Meudec

1 Introduction

1.1 Validation vs. Verification

2 Verification

2.1 Bugs

2.2 Debugging

2.3 Regression Testing

3 The Many Different Kinds of Verification Techniques

4 Software Testing: Dynamic Verification

4.1 Software Testing Efficiency at Finding Bugs

4.2 What is a Test?

4.3 Where do Test Cases Come From?

4.3.1 Black Box Test Cases

4.3.2 White Box Test Cases



4.3.3 Black Box vs. White Box Testing: Who's The Winner?

4.4 Unit vs System Testing

4.5 Measuring Testing Coverage: Coverage Metrics

5 Software Testing: Unit Testing Tools

5.1 Introduction

5.2 JUnit in Practice

5.3 Reflections

6 Conclusions on Testing 101

1 Introduction

Welcome to the least well-understood and least well-defined area of software development, the most hated area by most (not hated by me, though, as I find timeless, fundamental and unanswerable questions more interesting than latest technology fads).

This can be an exciting area to work in ([FYI] see, for example, [Unum job ad](#)).

All **agile developers must test their own code automatically** so that it can be repeated by others and form part of the regression testing suite.

There will always be two fundamental, unanswerable questions during software development:

- Will the product satisfy the customers/users? AKA the **validation** problem.
- Will the product always behave as we meant it to? AKA the **verification** problem.



1.1 Validation vs. Verification

This is a **very** important distinction. We need to do both verification and validation during software development.

Agile processes place a much stronger emphasis on validation than the Waterfall process.

Validation was always a weakness of the Waterfall: in theory, the stakeholders had very little scope to check the work in progress and were very often presented in the end with a product that they did not need, nor want, or that was totally unsuitable for them.

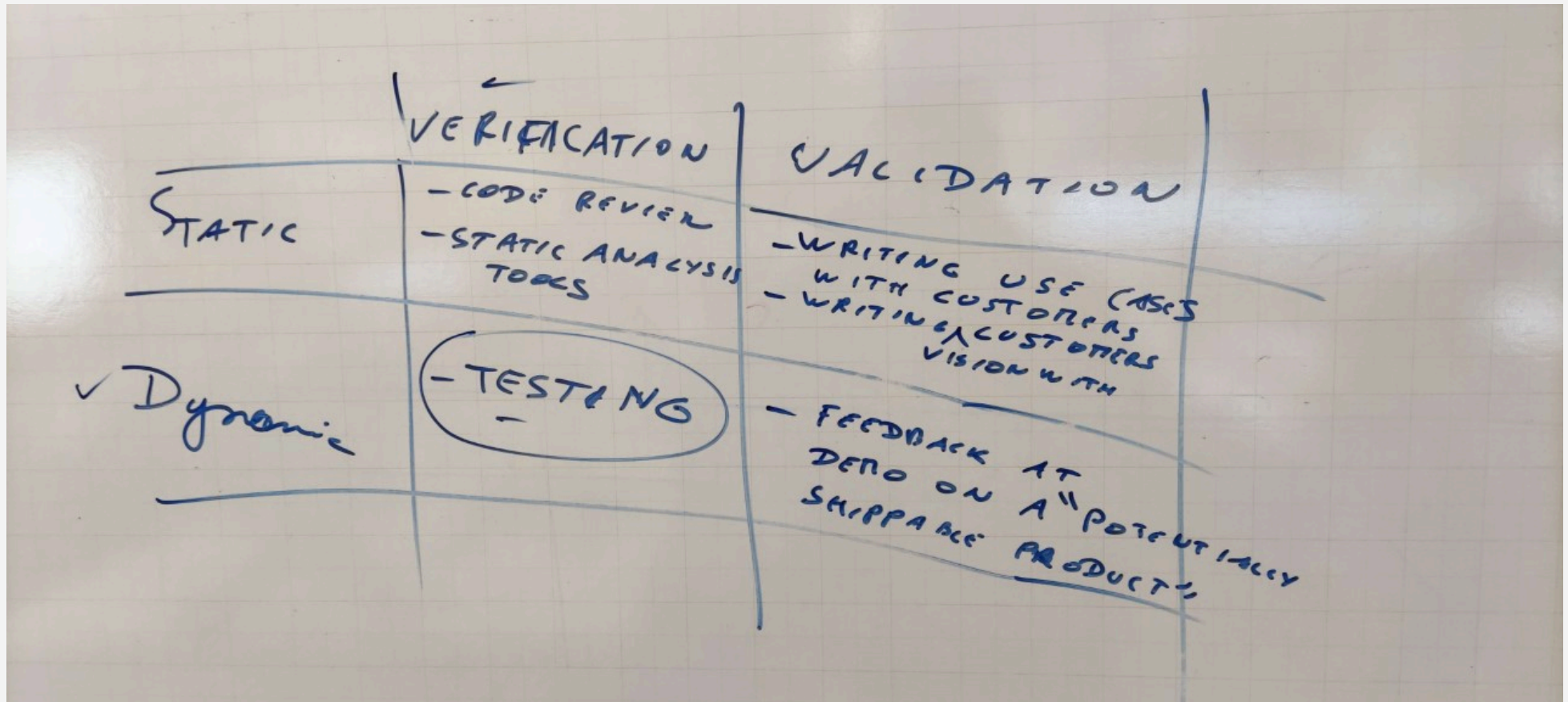
As we have seen, validation is the heart of evolutionary processes: the stakeholders guide the development of the product; they reject unnecessary features, suggest new ones, and improve existing ones and also validate the non-functional requirements as they **actually use the partially finished product at the end of each iteration**.

In agile processes, the customers/users are part of the team (this is a very strong statement if you know what a team is; in particular, a team is not a bunch of individuals; it is not a group).

We have covered validation activities: prototyping, teamwork with all stakeholders, writing use cases together, delivery of executable at each iteration, and active, regular, meaningful feedback seeking.

So, since we have already covered **validation** extensively, we will move on from validation and concentrate on **verification** in the rest of this tutorial.





Dynamic vs Static & Validation vs Verification Techniques

Remember, though:

- **You cannot do validation on your own:** you have to get out of the office (*'get out of the building'* in the lean startup mindset) and listen to real people;
- Validation activities ensure that whenever a customer with a family of 5 asks you for a people carrier for their holiday, you don't build them one of [these](#)... because although it may be a very nice product, it is not suitable for a family of 5...
- Validation activities try to answer the question: "*Are we building the right product?*".

- If you do not validate your product early enough (as mandated by agile) then very bad things can happen: the following video shows the developer on the left exposing their finished product for the first time to a user on the right: [1' video: late validation is no validation](#).



Can you explain the difference between verification and validation in your own words?

2 Verification

So what is verification?

Software verification activities try to ensure that the code will behave as we intend it to.

We all know that code does not always behave as we expect; see, for example, [Toyota's problem](#) with their software in 1.9 million Prius:

"In limited cases, the hybrid system might shut down, and the vehicle will stop, perhaps while being driven,"
Toyota said.

Code is just code, however, it behaves just as it is written... but as code becomes more complex, as more of it gets written and as it increasingly interacts deeply with millions of lines of other people's code, the actual code's behaviour is often quite different to what we expected... that's what we call **bugs**.

2.1 Bugs

Software bugs are faults, flaws, or errors in software that result in unexpected (different from the specification) or unanticipated (different from what would be expected) outcomes. Remember, the specification can be wrong, the software valid with respect to its specification, but a bug can still be detected...

Bugs take many different shapes, for example:

- bugs that crash the product (the Operating System may crash with it too!): that's a *nice* (unless it is safety or mission critical of course) bug as at least it is obviously a bug;
- bugs that make the product not behave as it should: e.g. in banking software, giving out the wrong monthly loan repayment figures for mortgages;

Bugs may occur in different ways:

- the bug occurs every time the software is run: that's *nice*, as it is easy to reproduce.
- the bug only occurs if specific actions are taken precisely in a specific order.
- the bug only affects some customers (software environment problems, configuration problems?);
- the bug only occurs sometimes, seemingly at random (C/C++ pointers problems?);

So verification is mainly about looking for bugs (“*Are we building the product right?*”).

But not only, as it is also verifying, for example:

- that the performance levels expected (and detailed, at least in the Unified Process, in the Supplementary Specification) are respected (e.g. 0.2 seconds maximum to store a fill-up with a photo in a bad network area and with a phone with known poor GPS functionalities);
- that a novice user can complete a given task under 2 minutes (verifying a usability requirement) (is this a validation or verification activity, though?);
- etc.

But mostly, yes, verification is looking for bugs.



2.2 Debugging

Once a potential bug has been found, it needs to be:

- confirmed: are you sure this is a bug?
- reproduced: can you find the minimal sequence of events that reveals the presence of the bug?
- localised: where in the code is the bug localised?
- Fixed: the code needs to change to remove the bug ([Rubber Ducking](#) may help: basically describe the problem as clearly as you can. Often, starting writing a post for Stackoverflow (where you will be told off for lack of clarity) does the job for me: by the time I have finished writing my post, I have fixed the problem in my head; no need to press send...;
- Add tests: add tests to the current test suite which would have detected the bug;
- retested (AKA **regression testing**): are you sure your code changes fixed the bug? Did you create new bugs by fixing one (i.e. did the code '*regress*'?)? Did fixing one bug allow other bugs, already present in the code, to become visible?

Debugging is mostly reproducing, localising and fixing bugs.

Debugging can be very time-consuming (most of the time, bugs can be fixed within minutes, and some will take a few hours, but it is also not unusual to spend days on a single bug) and frustrating.

Pointer-related bugs are notoriously difficult to reproduce, localise and fix (hence, most recent programming languages (Java, C#, Rust) do not allow pointers in the same way as C and C++ did).

2.3 Regression Testing

As mentioned, **regression testing** is the activity of re-applying previous tests **automatically** using tools to ensure that a bug fix (or code enhancement) has not broken the code.

In other words, regression testing tries to ensure that the modified code behaves as expected for all the previously applied tests (there could be 1000s) and that the code has not '*regressed*'.

Regression testing is an important technique because:

- introducing new bugs while trying to fix existing bugs, or generally modifying the code, is very, very common;



- The sooner a bug is detected, the easier it is to localise and fix (because the code is still fresh in your mind). An Agile saying is "*test early, test often, test realistically*".

Clearly, regression testing is only manageable whenever the tests can be run automatically using a software tool such as JUnit (for Java code), but more on this later: manually re-running 1000s of tests and checking the behaviour of the code just takes too long to be feasible.



3 The Many Different Kinds of Verification Techniques

So, how do we find bugs?

PS. Relying on your users to find bugs is typically not recommended in safety-critical software nor in business software...

Now, just to be clear. The “*running the code a few times to see if it works*” approach is not suitable in the real world. It might be OK for a coding assessment in college, but not when your software will be used by others and their business, personal data, or even life depends on your software...

Typically, to reach a level of reliability suitable for general release, the software has to undergo a lot of testing: about 100% of the initial development effort (less when using an agile process, because testing is so much more embedded in the entire software life cycle than in the Waterfall).



So, if it took you 3 months to develop your app, you can expect to spend another 3 months on verification tasks before a large-scale release.

Also, ..., believe it or not, **testing software** is only one way of looking for bugs in software (!!), there are many others.

In particular, there is a large family of verification techniques called **static verification techniques**. Unlike dynamic verification techniques (AKA software testing) a static verification technique does not require that the code be run to identify bugs.

Examples of static verification techniques include:

- **code inspections** (AKA code reviews, code walkthrough):
 - involve the very close, slow reading of code by developers trained in code inspections, other than the main author of the code;
 - bugs can be discovered (e.g. logical errors such as using an ‘and’ instead of an ‘or’ in Boolean expressions);
 - Very useful for junior developers as they learn from more experienced colleagues via feedback
 - Other characteristics of the code can also be noted for improvement (e.g. readability, coupling, cohesion), which may require [refactoring](#) (e.g. to reduce its 'technical debt').
 - A programming language-dependent checklist is available.
 - tool support, especially for distributed code reviews, is available (e.g. [Review board](#), [Crucible](#) or as [a basic extension for VSCode](#));
 - Code reviews can be very effective if performed formally or by people who care.
 - code reviews can be expensive due to the amount of time highly paid staff members have to devote to them.
- **static analysis tools**:
 - static analysis tools analyse source code and try to look for potential bugs;
 - note that, apart from looking for potential runtime errors (which, if they arise, are most likely due to bugs), most static analysis tools cannot find bugs since they have no knowledge of what the code is supposed to do: all they know is what the code does, not what the specification of the software is;
 - mostly static analysis tools look for “[code smells](#)” i.e. things that look suspicious, e.g.:
 - assigning the same variable twice without using it in between assignments, e.g.:

```
x = 6;  
//do more stuff without using x  
x = 7;
```

- unreachable code (AKA dead code), e.g.:

```
x = 1;  
if (x > 5) {//do things;}
```

- **duplicated code;**

- strongly typed programming languages (e.g. Ada, Java, C#) are easier to analyse than [weakly typed languages](#) (e.g. JavaScript, C, C++) and thus static analysis tools for these languages are more powerful and useful;
 - For example, in JavaScript, in an expression 'x+y', x could be a string and y could be an integer; the compiler/interpreter will not complain and evaluate the expression with often unwanted consequences. In strongly typed languages, adding an integer to a string is illegal. Weakly typed languages are very flexible, but require great care and additional checks.
- tools examples include:
 - [PMD](#) and its highly effective Copy/Paste Detector;
 - Lint: a whole range of diverse tools for various languages (including [Android Lint](#), which is part of Android Studio)
 - [SonarQube](#) used by UNUM



Let's Recap...a lot of new material introduced

4 Software Testing: Dynamic Verification

First of all, dynamic verification is the core of this tutorial.

Second of all, this is a very strange area of software development: it is fuzzy and full of potential traps (i.e. a good tester must have good technical skills, of course, but more importantly, one has to be very, very careful about testing):

- For example, **if it is very easy to test software against itself, which is a total waste of time** (more on this later);
- It is also recommended that other developers than just the author of the code be involved in the testing process: **can you tell why?**
- Finally, for now, if I give you some code and tell you to test it: what should be your answer?

These are not just mere musings: they are at the heart of what software testing is (and mostly, what software testing isn't). I'll make no apologies to emphasise this point again and again.

Thirdly, testing is the most widely used verification technique in the real world: some projects may not use code reviewers nor static analysis tools, but all use software testing as a means of trying to discover bugs.

4.1 Software Testing Efficiency at Finding Bugs

Testing is not a great technique for finding bugs: it is a bit like looking for needles in a haystack by examining a few individual straws... in the end, you don't know how needles remain even if you found many or none (same with bugs, you may have found many or none, but it does not help you answer the question: is there any bugs remaining in the product?)

The **input domain** of most software and even individual methods is so large (e.g. think of a method taking in an array of 10 integers: how many different inputs do you think that this method has?) that running a few tests (thousands, even) is, in theory, pretty hopeless.

Even if you try to do a very good job at testing the software:

- How do you know that you have?
- How do you know if there are remaining bugs or not?



- How do you know that another day of testing would not reveal important, serious bugs?

Mostly, none of these questions can be answered after **any** period of testing...

Here is a very famous quote (the **number one quote of software testing**) for you:

"Program testing can be used to show the presence of bugs, but never to show their absence!" Edsger W. Dijkstra, 1970

[An alternative phrasing is *"a passing test tells you nothing, but a failing test shows something is broken"*.]

This is a pretty damning condemnation of software testing as a verification technique... no amount of testing (apart from exhaustive testing, perhaps, but that is infeasible for the vast majority of software) can guarantee bug-free software!

And yet it is still the most common form of software verification performed in the industry...

Some bugs remain undiscovered for years even in heavily used products: FYI, [see this 24-year-old bug in the Linux Kernel implementation of the TCP protocol](#).

A common suggestion to improve matters in view of reducing bugs in released software, and developing software quickly, is to use a mixture of static and dynamic verification techniques:

- Use static analysis tools suitable for the target programming language from day one, every day on all the code (because they are cheap and fast at finding potential problems);
- have means to test the software automatically (see later), so that you can *test early, often and realistically*;
- perform code reviews once per iteration, for example;





Let's review.

4.2 What is a Test?

Good question.

Informally, a test can be many things, but it must allow the software or code to be run either automatically or manually:

- it can describe a series of actions at the GUI level: [see this real-life example](#) [system level, manual test];
- it can contain actual inputs to a specific method:

```
addVehicle("00 CW 909", "Ford", "Ka", "Red"); [unit level, can be run automatically];
```

These are all tests... but they are not useful at all.

In fact, they are useless: I would even go as far as to say that these 'tests' are dangerous. **Why?**

You could run hundreds of similar tests, and you would pretty much waste your time: you would test the software alright, but what for? To detect app crashes? Maybe. To detect runtime errors? If you are lucky, perhaps. But you will not detect any other kinds of bugs with tests like these...

Because you see, when you test something, anything, software-related or not, you must know what it is that you are testing, i.e. what it is that you expect the software to do!

So a test that does not describe what the **expected behaviour** of the software is, is useless (dangerous even because you think you

are testing something, but you're not really).

So, if a 'test' is something that allows us to run the software or specific code in a particular manner, it is pretty useless without a clear description of how the software is supposed to behave according to its specification.

Formally, a **test case** is a test and **a description of the intended behaviour according to its specification**. This is a central concept.

So, dear reader, if I give you some commented code, **can you test it (if yes, why? If not, why not?)**? [FYI, here are [examples of interesting comments](#) found in code]

Retaking the same example as earlier, here are examples of **test cases**:

- [real-life example](#) [system level, manual test case];
- `addVehicle("00 CW 909", "Ford", "Ka", "Red")== rowID and query("VehicleTable",rowID).equal(Vehicle("00 CW 909", "Ford", "Ka", "Red"))` [unit level, a test case can be run automatically];

Note how the last example above can be run automatically and also how the actual behaviour of the code can be recorded automatically against the expected behaviour of the code: inserting a new vehicle in the database should mean that the same vehicle can be retrieved from the database afterwards. Obviously, if a bug does exist in the code that inserts a vehicle, it will be detected automatically: that's the magic of automatic software testing using **test cases** that can be run automatically.

4.3 Where do Test Cases Come From?

So, if a **test case** is a combination of inputs and expected outputs from the specification, where do we get test cases?

Obviously, we can create random test inputs easily (at least at the method/unit level), but these are unlikely to have a high probability of finding a bug...

Because, remember, we are doing verification here, **we are bug hunters**, we want to break the code, we want the code to misbehave: in the sea of possible inputs, we want to find the ones that are the most likely to reveal the presence of a bug.



So yes, random testing is a possibility (but don't forget we also need to derive the expected output of every test input by thinking about the expected behaviour of the code: and that takes time to do, it is a manual, human process, it is expensive. "Running 1000s of tests" also means finding the expected behaviour for these "1000s of tests", which would be very time-consuming). Random testing is possible in simple circumstances, but it is mostly a technique that increases confidence in the code under test: it is not a good way of finding bugs (especially when running a single test takes over an hour, as in our previous real-life example).

There are two main, complementary **families** of techniques to generate test cases:

- Black Box Test Cases;
- White Box Test Cases.



Let's pause here

4.3.1 Black Box Test Cases

Black Box testing involves testing the software using test cases generated using Black Box test cases generation techniques.

A Black Box test case is composed of **test inputs generated solely from the specification of the software** (e.g. from a use case). The code is not needed to generate black box testing, so:

- as soon as, for example, a use case is written, black box test cases can be generated;
- Black Box tests can be identified, created and written down before the code is available.

Of course, later on, the code needs to be available to actually run the tests...

Black box tests can be written by different people at different times, e.g.:

- early on, by high-level managers, using use cases: these test cases will be typically for system-level testing that are hard to run automatically, they will test the overall behaviour of the product;
- Later on, programmers can use the design: these test cases can be used for unit-level testing that can be automated, and they will test the behaviour of individual methods.

Remember, though: we are trying to find bugs! So the tests should not just be “*middle of the road tests*” where things are unlikely to go wrong; we want things to go wrong!

Often, tests are taken at the **boundary of input domains** (e.g. if the input should be between 0 and 100 and generate an error otherwise, then some tests should include -1, 0, 1, 99, 100, and 101). The same principle can be applied to select inputs that generate output values at the **boundary of output domains**. (e.g. if the output is the index in an array where an element being searched is, then tests should be generated so that the output is the first element in the array, the last, and also when there is no such index).

So, using our knowledge of the specification (whatever shape it may be in), we devise scenarios, or inputs, that are likely to cause problems for the code... and we also describe the expected behaviour of the software/code according to that same specification to obtain test cases that are likely to reveal the presence of bugs.

Later on, whenever the code becomes available, these Black Box test cases are run, and the code's behaviour is checked against the expected behaviour described in the test cases.



4.3.2 White Box Test Cases

White Box testing involves testing the software using test cases generated using White Box test case generation techniques.

White Box testing techniques use characteristics of the code under test to generate the test inputs part of test cases (as always, the expected output/behaviour of the code under test **must come from the actual product's specification**).

For example, if the code contains an if statement involving an input variable, two test inputs can be generated: one that will make the Boolean condition of the if statement true during actual execution and one that will make it false. Thus, during testing, the actual logic of the if statement will be indirectly checked by the two tests.

As in Black Box techniques, some White Box techniques generate tests on “the boundary” of the input domain of a method. So, for example, in `if (x >= 5) {...}`, several test inputs should be constructed where `x` at that point should be:

- -100 (e.g.) because negative numbers are often overlooked;
- 0 because 0 often causes problems;
- 4 because it is right at the true/false boundary;
- 5 because it is right at the true/false boundary;
- 6 because it is right at the true/false boundary;
- 42424242 (e.g.) because it is a large positive number.

As you can see from this simple example, many tests can be generated... each one of these would also require the expected code behaviour to be extracted from the specification, which is a time-consuming and error-prone process.



4.3.3 Black Box vs White Box Testing: Who's The Winner?

Nobody.

No winner: you must use both techniques... Here is why...

High-level (e.g. from use cases), black-box tests are very valuable as they have the potential to identify bugs that may be overlooked otherwise... Think of the following, very common, scenario:

1. the specification (Vision, detailed use cases, supplementary specification, etc.) mentions that the software should do A, B and C;
2. the design takes place: features A, B and c are designed;
3. coding takes place: features A, and B are coded;
4. the testing takes place;

What happens at step 4 if only white box testing is used?

On the other hand, just relying on tests generated using Black Box techniques is not a good idea because not all the code may be executed during testing: it is very common that Black Box-generated tests do not execute every method in the code, never mind every line of code...

Would you step on a plane if some of the engine controller's code had never been executed at least once during testing?

So, relying on a single kind of testing is not recommended; both families of techniques have advantages and disadvantages: Black Box and White Box testing techniques are complementary.



4.4 Unit vs System Testing

We mentioned Unit testing and System testing in this tutorial without really defining them.

Unit Testing is about testing an individual method, or small groups of methods, on its own: i.e. a small unit of code. Testing a method on its own is typically not achievable from the GUI: instead, additional code has to be written that calls the method with specific arguments (the test input); this additional code is called a **test driver**. A test driver sets up the necessary environment for the method under test to be run (e.g. other objects that the method under test relies upon, the method arguments themselves). The method's behaviour should be checked against its expected behaviour according to the specification. White box-generated tests are usually at the unit level, i.e. for an individual method or small groups of methods. Actually running **Unit Tests** can usually be automated (that's the easy part; the hard part is designing test cases) but it requires writing a lot of additional code to setup the environment (to create mock objects and resources as part of the test driver, before calling the actual method under test with the correct arguments, and finally also to check the expected behaviour of the code as extensively as possible after the method under test returns). All this extra code creates a kind of scaffolding around the code under test: often there is more scaffolding than actual building, i.e. more testing code than actual code...



[[Zagreb's Cathedral and its scaffolding](#): a metaphor for testing code surrounding actual code]

Integration testing must follow Unit Testing: when all the classes/units are put together (e.g. merging code with team workers), it is necessary to verify that the different units work as expected together. This is often done in the cloud repository (e.g. using GitHub Actions) as part of a DevOps.

System Testing, on the other hand, is about exercising the software from the GUI, or at least from a high level (such as at the API level): it is not about targeting a particular method but rather the overall behaviour of the code. **System Testing** is harder to automate.

Automatic Testing means the ability to execute the software/code under test automatically using specific tests, the ability to collect its **actual behaviour** and the automatic comparison of the **expected behaviour** against the actual behaviour for eventual automatic bug reporting.

[FYI [extreme regression testing with Google Cars](#)]

4.5 Measuring Testing Coverage: Testing Coverage Metrics

It is possible to measure how much of the code is executed by a single test, or more commonly, by a set of tests often called a **test suite**.

For example, we can measure the percentage of a method's code statements executed by a test suite: it could be 20%, 40% or 100%.

In general, when we measure something in software development, it is called a [metric](#); metrics attempt to measure such varied things as code complexity (why would we want to measure that?), level of coupling, the productivity of programmers, etc.

Code Coverage Metrics (coverage metrics for short) measure the amount of code executed during testing, including, in **order of increasing strength**:

- **statement coverage**: what a statement is, is language-dependent, but any atomic construct (such as an assignment) ending with a semicolon is a good approximation;
- **branch coverage**: a branch is a code construct that changes the path of execution of the code, such as an if statement, a



loop or a switch statement (note that a switch statement may create many more than 2 branches);

- **decision coverage:** a decision is a boolean expression e.g. the right-hand side of this assignment is a decision

```
target = distance < 1000 and size > 50;
```

Decisions may or may not create a branch. Decision coverage measures how many of the possible true and false decision outcomes have been exercised during testing.

- **condition coverage:** a condition is a Boolean atomic component of a decision, e.g. in the decision:

```
distance < 1000 and size > 50
```

, there are two conditions: `distance < 1000` and `size > 50`. Condition coverage measures how many of the possible true and false conditions' outcomes have been exercised during testing. Every condition within a decision must be executed for true and false.

There are many other coverage metrics. [FYI] The most difficult to understand and the most difficult to reach 100% is the [MC/DC coverage metric](#) (MC/DC is a bit like Quantum Physics: “*if you think you understand it, you don't*” Richard Feynman). MC/DC is the standard coverage metric in many safety-critical applications, such as avionics software (FYI, see [NASA's tutorial on MC/DC](#)).

So, a goal set by a project manager might be for the testing process of a project to “*achieve 100% statement coverage*” (necessary but not sufficient and misleadingly reassuring).

Coverage can be measured using software tools. These same tools can also indicate which statement, or decision etc., has not been fully covered during testing: they can help testers to achieve a higher level of coverage by devising specific tests to target the uncovered parts of the code as indicated by the tools.

Here is a typical screenshot from a code coverage analysis tool:



Java - CursorableLinkedList.java - Eclipse SDK

File Edit Source Refactor Navigate Search Project Run Window Help

JUnit Finished after 34,898 seconds

Runs: 13009/13009 Errors: 0 Failures: 0

Failures Hierarchy

- junit.framework.TestSuite
 - junit.framework.TestSuite
 - TestBagUtils
 - org.apache.commons.collections.TestClose
 - org.apache.commons.collections.TestColle
 - TestBufferUtils
 - TestEnumerationUtils
 - org.apache.commons.collections.TestFact
 - TestListUtils
 - TestMapUtils
 - org.apache.commons.collections.TestPrec
 - TestSetUtils
 - org.apache.commons.collections.TestTran
 - TestArrayStack
 - TestBeanMap
 - org.apache.commons.collections.TestBina
 - TestBoundedFifoBuffer
 - TestBoundedFifoBuffer2
 - TestCursorableLinkedList
 - TestDoubleOrderedMap
 - org.apache.commons.collections.TestExte
 - TestFastArrayList
 - TestFastArrayList1
 - TestFastHashMap
 - TestFastHashMap1
 - TestFastTreeMap
 - TestFastTreeMap1

CursorableLinkedList.java

```

public boolean addAll(int index, Collection c) {
    if(c.isEmpty()) {
        return false;
    } else if( size == index || size == 0) {
        return addAll(c);
    } else {
        Listable succ = getListableAt(index);
        Listable pred = (null == succ) ? null : succ.prev();
        Iterator it = c.iterator();
        while(it.hasNext()) {
            pred = insertListable(pred, succ, it.next());
        }
        return true;
    }
}

```

Problems Javadoc Declaration Console Coverage

TestAllPackages (31.10.2006 15:04:14)

Element	Coverage	Covered Lines	Total Lines
java - commons-collections	79,5 %	10927	13738
org.apache.commons.collections	74,1 %	3842	5183
ArrayStack.java	86,5 %	32	37
BagUtils.java	86,7 %	13	15
BeanMap.java	72,4 %	155	214
BinaryHeap.java	87,6 %	127	145
BoundedFifoBuffer.java	93,2 %	82	88
BufferOverflowException.java	55,6 %	5	9
BufferUnderflowException.java	88,9 %	8	9
BufferUtils.java	30,8 %	4	13
ClosureUtils.java	93,9 %	31	33
CollectionUtils.java	92,4 %	293	317
ComparatorUtils.java	8,6 %	3	35
CursorableLinkedList.java	85,4 %	444	520

Failure Trace

Writtable Smart Insert 149 : 28

[From <http://www.eclEmma.org/> “Java code coverage for Eclipse”]

It is important to understand that “*achieving 100% statement coverage*” does not mean very much as far as the quality of tests applied is concerned. For example, in the code below:

```
a;  
if (b) {  
    c;  
}  
d;
```

any test which makes `b` true during execution will achieve 100% statement coverage... This would be achieved without ever testing the code’s behaviour whenever `b` is false... Nobody would call that suitable.

Even “*achieving 100% branch coverage*” is very often quite poor, for example, in

```
if (A or (B and C)) {  
    do_something;  
} else {  
    do_something_else;  
}
```

it is quite possible to achieve 100% branch coverage without ever checking the code’s behaviour whenever `B` is true (or even without `C` being ever evaluated at all in languages with short-circuit evaluation of Boolean expressions !).

Clearly, one has to be careful with coverage metrics... *100%* of whatever always sounds attractive, but it does not always amount to much in terms of the code’s behaviour being tested.

Further, even executing every single little bit of the code does not mean that you have tested the code properly, nor that there are no bugs remaining in your code: there is no straightforward relationship between the amount of code tested and the number of remaining bugs in the code. In fact, increasing the level of testing coverage does not necessarily reduce the likelihood of a bug in



your code.

Here is a simple example:

```
x = 1 / (y - 42);
```

This statement could be executed with a thousand different inputs, or a million, if you wish, without ever finding the underlying division by zero bug...

Understand, therefore, and remember, that testing coverage metrics are pretty **useless**, actually.

They are nonetheless widely used...

Why?

Well, that too is easy to understand:

- they sound good: 100% of anything always sounds good...;
- they are reassuring;
- They are easy to collect;
- And finally, for the killer reason... would you step into an aeroplane if you knew that some portion of the code running within it had never been executed during testing? ... ([FYI] Boeing's new 787 Dreamliner runs 6.5 million lines of code to keep it in the air, and your car is not far behind).

So, achieving high code coverage during testing is **necessary but not sufficient**.

In practice, these tools can be used to complement black-box tests to ensure that coverage is achieved. For example, exception handling code is rarely well covered by black-box tests: complement them using white-box tests to achieve coverage.

Finally, one can understand why some say that high levels of testing coverage reflect more the effort put into the testing process than its quality...





5 Software Testing: Unit Testing Tools

5.1 Introduction

JUnit is a **Unit Testing Framework** for Java, originating from [Kent Beck](#) (a very influential software engineer), it is part of the **xUnit** family of testing tools (similar tools for different programming languages, such as CUnit, CppUnit, do exist).

JUnit allows test cases to be executed by:

- calling the method under test automatically with the test input;
- checking the code's **actual behaviour** against its **expected behaviour** and generating an error report in case of divergence.

JUnit can also be used in a Test-Driven Development (TDD) approach, i.e. one where developers write test cases prior to writing the code ... this is an [eXtreme Programming](#) practice.

[FYI, the [JUnit GitHub repository](#)]

JUnit and similar unit testing frameworks are ideally suited to [regression testing](#) because they entirely automate the running of the code under test with specific test inputs, the gathering of the code's actual behaviour and the generation of bug reports if that behaviour diverges from the expected behaviour as described in the test cases.

Of course, this automation is not always easy to achieve:

- A lot of effort must go into writing the test cases in a way the testing tool understands.



- The main problem of coming up with the test cases in the first place, using either Black Box or White Box techniques, is not addressed by these tools.

5.2 [FYI] JUnit in Practice

JUnit is part of Android Studio, but here I want to show how JUnit can be used to test plain Java. Unfortunately, Android Studio cannot run plain Java easily. So we will switch to the IntelliJ IDE to show the basic usage of JUnit on plain Java.

The specification of the method multiply is:

given two integers x and y it returns x multiplied by y. Both inputs must be less than 1000 or an `IllegalArgumentException` should be raised.

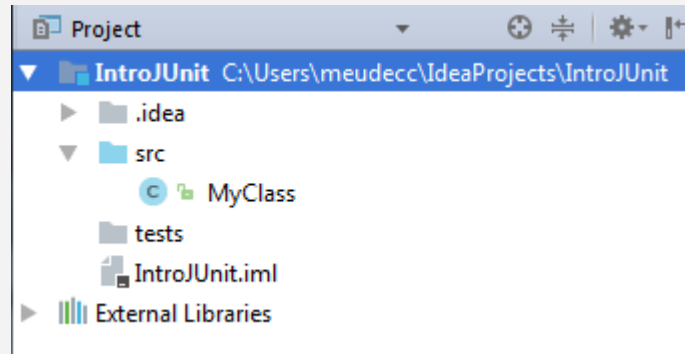
[FYI [JUnit on Eclipse](#)]

1. Install the **IntelliJ IDE** if you have no version of it already installed: [download link](#) (the community version is fine).
2. Create a new Java Project in IntelliJ entitled `IntroJUnit`.
3. Create a new class called `MyClass` in the `src` folder such as

```
public class MyClass {  
    public int multiply(int x, int y) {  
        // the following is just an example  
        if (x > 999) {  
            throw new IllegalArgumentException("X should be less than 1000");  
        }  
        return x / y;  
    }  
}
```

4. Right-click on the project name and create a new directory called `tests` to contain your tests, as in

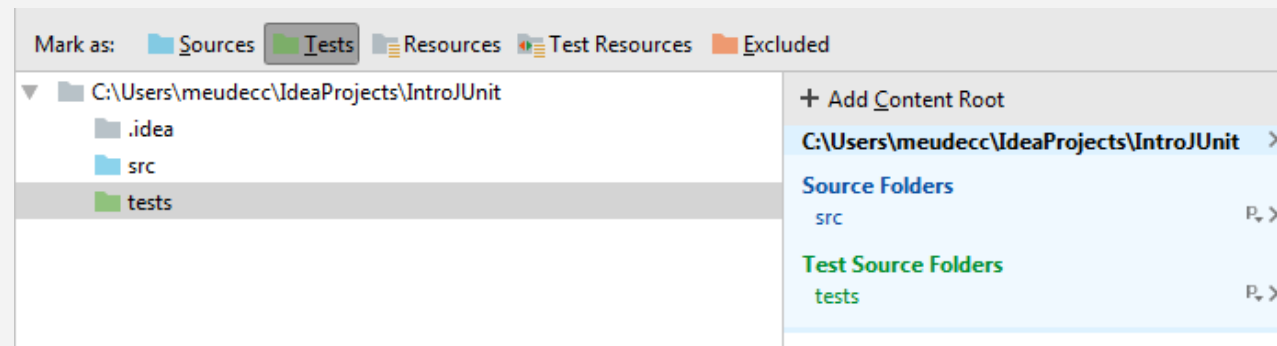




Tests Directory created

5. Tell IntelliJ that this new directory is for tests by
 - selecting File->Project Structure
 - select Modules
 - select Sources tab
 - select your tests folder in the Sources tab
 - and select the Tests button

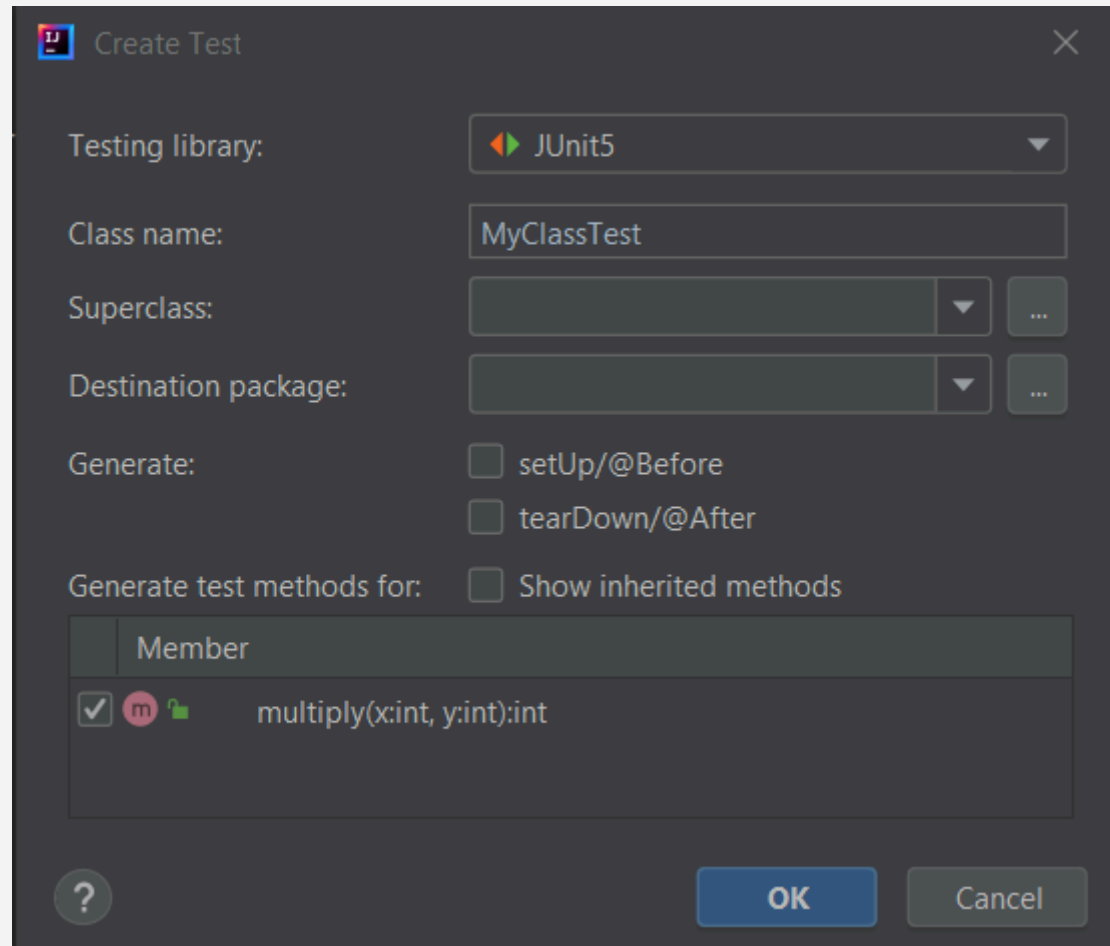
IntelliJ should now know that this folder will contain your test sources, as in



Test Source folder chosen

6. Click on MyClass in the code window and press Alt+Enter

7. Select Create Test
8. Choose JUnit5 as Testing Library
9. Select Fix to add JUnit5 library to the project (this may require downloading)
10. Select the multiplication method as in



IntelliJ JUnit setup

12. Press OK

13. You can now create tests for the multiply method. Based on the specification, 3 tests have been created.

Test#	1	2	3
Inputs	(1, 5)	(-2, 5)	(0, 0)
Expected Output	5	-10	0

editing the default template created as in:

```
public class MyClassTest {
    @Test
    public void multiplyBasic() throws Exception {
        MyClass tester = new MyClass();
        assertEquals(5, tester.multiply(1, 5));
    }

    @Test
    public void multiplyNegative() throws Exception {
        MyClass x = new MyClass();
        assertEquals(-1, x.multiply(-2, 5));
    }

    @Test
    public void multiplyBy0() throws Exception {
        MyClass x = new MyClass();
        assertEquals(0, x.multiply(0, 0));
    }
}
```

14. Note in the above that in `assertEquals(5, tester.multiply(1, 5));` the expected result, according to the specification, is the first argument
15. Run the tests by right-clicking anywhere in the file and selecting Run
16. If a test fail, ask yourself why:
 - Is it the test that is wrong? [possible]
 - Is it the code that is wrong? [also possible]

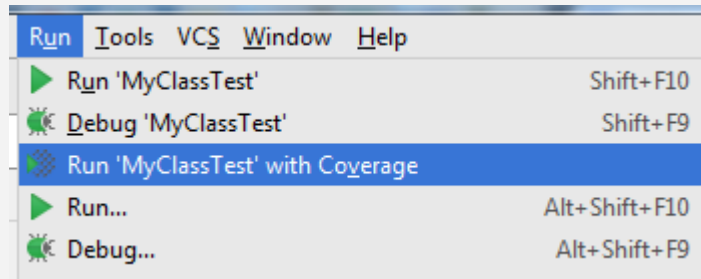
Fix the issue appropriately and rerun the test until the test succeeds

Now generate **black-box tests** for that simple example, knowing that the specification says that the method should multiply two integers together and return an error if the first argument is greater than 999:

- think of boundaries from the specification to cover the behaviour described in the specification completely.
- Write the tests using JUnit
 - by the way, this is how you state in JUnit that you expect an exception:

```
@Test
public void multiplyGreaterThan999WithMaxInt() {
    MyClass tester = new MyClass();
    IllegalArgumentException thrown =
    Assertions.assertThrows(IllegalArgumentException.class, () -> {
        tester.multiply(Integer.MAX_VALUE, 0);
    });
}
```

- run your tests and check the test run report
- You can also run the test with coverage:



Using IntelliJ to gather test coverage metrics

[FYI, [a more thorough tutorial](#)]

A more interesting example:

Create and run tests to achieve **100% branch coverage** of the following code (which does not do anything useful so the expected result is difficult to describe: this is just an exercise)

```
public int whatever(int x, int y) {  
    int a;  
    int b;  
    if (x + y > 42) {  
        a = x;  
        b = 0;  
    } else {  
        a = y;  
        b = 5;  
    }  
    if (a > 10) {  
        b = b*2;  
    } else {  
        b = b-2;  
    }  
    return a+b+x+y;  
}
```



[FYI] Final exercise using multiple objects:

1. clone the repository <https://github.com/echancrure/testing2017.git> using IntelliJ (if using a different IDE the projects need to be converted first)
 2. explore its document in the doc folder
 3. explore the code
 4. write tests for `Eights.getPlayerName()` and `Eights.setPlayer()`
 5. [FYI bonus question: how would you test the `Eights.deal()`?
 - a. What is the problem with testing `deal()`?
 - b. Why would it be important to test it?
 - c. What would be your advice?
-]

5.3 Reflections

Please take a moment to reflect on the power of such testing frameworks.

As long as

- You are disciplined and take time to generate Black and White box test cases.
- systematically write them down in the way the tool prescribes;
- run the tests early, often and realistically;
- keep the tests up to date (e.g. if new functionality is added or if the behaviour of a method is extended).

You can gain huge benefits from using such a testing framework:

- newly-introduced bugs can be detected immediately;
- Every day, as you extend or modify your own code, or more importantly, someone else's code, as is the rule in agile processes, you can make progress in the confidence that you are not breaking any of the old code and that your new code works as expected.



Because the running process is automatic, test suites can be run several times a day: this does not happen with manual testing.

Yes, it is an effort to write all these tests, but it is an investment that repays itself very quickly with better code quality, fewer debugging hours, and easier code expansion and maintenance.

In all, it makes you work smarter, not harder.

This is getting even more essential in the era of gen-AI generating a lot of code: yes, AI can generate testing code too, but who ultimately has the responsibility for ensuring fulfilment of all the requirements (yes, that includes security and reliability requirements): **it's you, the AI-enabled software engineer.**

6 Conclusions on Testing 101

We have covered a lot in this tutorial, and it may be beneficial to reread it for the information to sink in.

Testing is a fundamental problem in software engineering and will always be: it will never be solved by a magic tool.

It is a challenging area that is often counterintuitive; for example, increased testing coverage does not necessarily imply a higher likelihood of finding a bug.

[FYI, see this [recent research paper](#) (Inozemtseva, Laura, and Reid Holmes. "*Coverage is not strongly correlated with test suite effectiveness.*" Proceedings of the 36th International Conference on Software Engineering. ACM, 2014) on the effectiveness of coverage metrics at judging the quality of test suites. Retrieved from a search on [Google Scholar](#)]

It is also an area that requires discipline, and smart people: too often developers go headlong into coding with a poor design and no testing strategy, full of enthusiasm, they work very, very hard, do overtime and work weekends. The result: very poor software products, full of bugs, are very expensive and time-consuming to fix, resulting in poor quality, over-budget, and late projects, which are the norm in the software industry.

Be better than this.

Be a professional.



Work smarter, not harder.

Questions?

